From Devboard to Production

A Real Zephyr Case Study — Live Meeting 2





The Story



Iomico prototyped an optical RPM counter to measure motor RPM on a test bench. To move fast, the team started with what they had on hand: an **FRDM-K64F** and a **CNY70** optical sensor aimed at a slotted disc.
It worked well — so for production they moved to something more compact: their own board around the **MCXN236**. The application code never noticed the change.



What This Demo Ties Together



- **Kconfig** (L03) — software parameters: pulses/rev, log window
- **Devicetree + overlays** (L04) — the sensor as a hardware node, `pulse-in` alias
- **Custom boards** (L05) — from "patch on someone else's board" to "we own the whole tree"

A new use case (pulse counting + RPM), built with pieces you already know.

Objectives

- See devicetree **overlays** used for real prototyping, not just LEDs
- See a **custom board** replace a dev board with zero app code changes
- See **Kconfig** driving a board-specific default in production
- Build a mental model: **devboard + overlay → custom board**



Act 1 — Prototype

The Application Code

Identical on both boards — it only knows about a devicetree alias.

```
#define PULSE_NODE DT_ALIAS(pulse_in)

static const struct gpio_dt_spec pulse_in =
    GPIO_DT_SPEC_GET(PULSE_NODE, gpios);

/* ISR increments pulse_count on every rising edge */

LOG_INF("pulses=%u  ->  RPM=%d", pulses, (int)rpm);
```

No hardcoded pins. No `#ifdef BOARD_*`. Pulses/rev and log window come from `CONFIG_APP_*` (Kconfig).

The Overlay: `boards/frdm_k64f.overlay`

We model the CNY70's digital output as a button — same `gpio-keys` pattern from the Devicetree lecture.

```
/ {
    aliases {
        pulse-in = &cny70_pulse;
    };

    pulse_sensor: pulse_sensor {
        compatible = "gpio-keys";

        cny70_pulse: pulse_0 {
            gpios = <&gpioc 6 (GPIO_ACTIVE_LOW | GPIO_PULL_UP)>;
            label = "CNY70 pulse";
        };
    };
};
```

Live: Build + Flash

```
west build -b frdm_k64f app
west flash
```

- Generate pulses (spin the disc, or tap the pin by hand)
- Watch RPM logs on the console

Then open the generated devicetree and find the merged node:

```
build/zephyr/zephyr.dts
```



Interlude — Kconfig Live



Tune Behavior Without Touching C




```
west build -t menuconfig
# APP_PULSES_PER_REV, APP_LOG_PERIOD_MS
west build
west flash
```

Same hardware, same binary structure — different RPM math and log cadence, purely from Kconfig.

Custom Board: `rpm_counter` (MCXN236)



Act 2 — Production

The Story Turns

The team is done prototyping — no more dev board in the final product.
They design their **own PCB** around the MCXN236.

```
boards/
└── rpm_counter/
    ├── board.yml
    ├── Kconfig.rpm_counter
    ├── Kconfig.defconfig
    ├── rpm_counter_defconfig
    └── rpm_counter.dts
```

Same minimal layout as "Common to Both Methods" from the Custom Boards lecture.

```
west build -b rpm_counter app   # west finds it with no extra flags
```

The Key Difference

Zephyr®

Prototype

```
/* boards/frdm_k64f.overlay */
/ {
    pulse_sensor: pulse_sensor {
        compatible = "gpio-keys";
        cny70_pulse: pulse_0 {
            gpios = <&gpioc 6 ...>;
        };
    };
};
```

A **diff** on someone else's board.

Production

```
/* rpm_counter.dts */
#include <nxp/mcx/nxp_mcxn23x.dtsi>
/ {
    pulse_sensor: pulse_sensor {
        compatible = "gpio-keys";
        cny70_pulse: pulse_0 {
            gpios = <&gpio0 6 ...>;
        };
    };
};
```

The **base definition** — our own board.

Live: Build + Flash — Same App

```
west build -b rpm_counter app
west flash
```

`app/src/main.c` — **zero lines changed**.

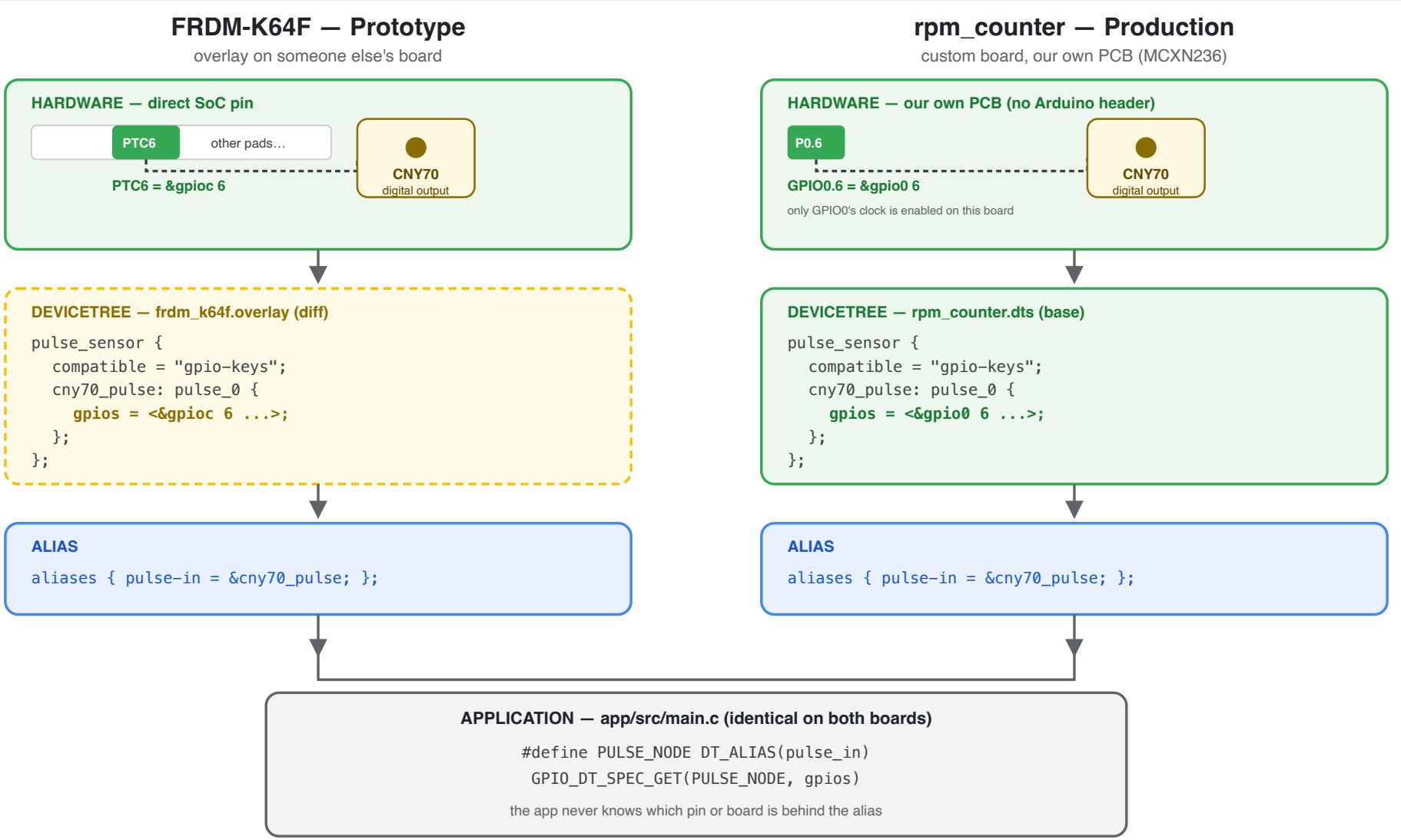
Bonus (if time allows): `Kconfig.defconfig` sets a different default for `APP_LOG_PERIOD_MS` on this board — Kconfig board-specific defaults.

```
if BOARD_RPM_COUNTER
config APP_LOG_PERIOD_MS
          default 250
endif
```



Closing

Devicetree ↔ Hardware — The Whole Picture





Overlay vs. Custom Board



	Overlay	**Custom board**
Whose base hardware is it?	Someone else's (official dev board)	Yours
What file do you touch?	`.overlay` (a diff)	The full `.dts`
When to use it?	Prototyping, adding a peripheral to an existing board	Production, your own PCB
App code impact	None	None

Takeaways

- Devicetree **aliases** are what let application code survive a hardware platform change
- An **overlay** is a patch; a **custom board** is ownership of the whole tree — same mental model, different scope
- **Kconfig** stays orthogonal to both — software knobs, not hardware wiring

Thank You

Questions?



Resources

1. Devicetree Guide
2. Zephyr Board Porting Guide
3. Kconfig Reference
4. Full demo sources: `zephyr-fundamentals-lm2/`